

# Lab IPsec IKEv2/ESP

Túnel IPsec con strongSwan sobre Docker

---

|                     |                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------|
| <b>Autor:</b>       | Diego Medardo Saavedra García                                                               |
| <b>Programa:</b>    | Máster en Ciberseguridad                                                                    |
| <b>Universidad:</b> | Universidad Complutense de Madrid                                                           |
| <b>Fecha:</b>       | Junio 2026                                                                                  |
| <b>Repositorio:</b> | <a href="https://github.com/statick88/ipsec-lab">github.com/statick88/ipsec-lab</a>         |
| <b>Docker Hub:</b>  | <a href="https://hub.docker.com/r/statick/ipsec-lab">hub.docker.com/r/statick/ipsec-lab</a> |

# Contenido

---

1. Resumen Ejecutivo
2. Objetivos
3. Arquitectura del Laboratorio
4. Metodología de Implementación
5. Configuración y Algoritmos
6. Resultados y Evidencias
7. Validación con Wireshark
8. Automatización y CI/CD
9. Conclusiones
10. Referencias

# 1. Resumen Ejecutivo

Este informe documenta el diseño, implementación y validación de un laboratorio de seguridad IPsec que demuestra el funcionamiento de túneles IKEv2 con encapsulación ESP entre dos gateways■ados mediante network namespaces en Docker.

El laboratorio utiliza **strongSwan 5.9.5** sobre **Ubuntu 22.04**, con algoritmos de cifrado AES-CBC-256 para confidencialidad y HMAC-SHA-256-128 para integridad. La autenticación se realiza mediante Pre-Shared Key (PSK), y todo el entorno se ejecuta en un único contenedor Docker con privilegios para operaciones de namespace.

Los resultados incluyen: túnel IKEv2 establecido, tráfico ESP cifrado verificado, paquetes capturados y decodificados en Wireshark, evidencia académica completa con metodología, y un pipeline CI/CD automatizado con tests.

| Componente          | Estado        | Detalle                            |
|---------------------|---------------|------------------------------------|
| IKEv2 SA            | ✓ ESTABLISHED | AES_CBC_256 / HMAC_SHA2_256_128    |
| CHILD SA (ESP)      | ✓ INSTALLED   | Modo tunnel, SPI 0x12345678        |
| Tráfico cifrado     | ✓ Verificado  | ICMP ping entre subnets protegidos |
| Captura pcap        | ✓ Generada    | ESP + ICMP decodificado            |
| Wireshark keys      | ✓ Exportadas  | Formato CSV compatible             |
| Tests automatizados | ✓ 13/13       | Unit + E2E + Security              |
| CI/CD Pipeline      | ✓ Funcional   | GitHub Actions (6/7 jobs)          |
| Docker Hub          | ✓ Publicada   | statick/ipsec-lab:latest           |

## 2. Objetivos

---

### Objetivo General

Demostrar el funcionamiento de un túnel IPsec IKEv2 con encapsulación ESP en un entorno controlado, validando los algoritmos de cifrado, integridad y autenticación utilizados en las comunicaciones seguras entre dos gateways.

### Objetivos Específicos

- Configurar dos gateways IPsec (east/west) utilizando network namespaces aislados
- Establecer un túnel IKEv2 con autenticación PSK y algoritmos AES-CBC-256/HMAC-SHA-256
- Validar el encapsulamiento ESP en tráfico entre subnets protegidos
- Capturar y decodificar paquetes ESP utilizando Wireshark/tshark
- Generar evidencia académica completa con metodología documentada
- Automatizar el entorno mediante Docker y scripts de validación
- Implementar pipeline CI/CD con tests automatizados y escaneo de seguridad

### 3. Arquitectura del Laboratorio

El laboratorio se ejecuta en un único contenedor Docker con privilegios, utilizando network namespaces para simular dos gateways IPsec independientes conectados por un par veth.

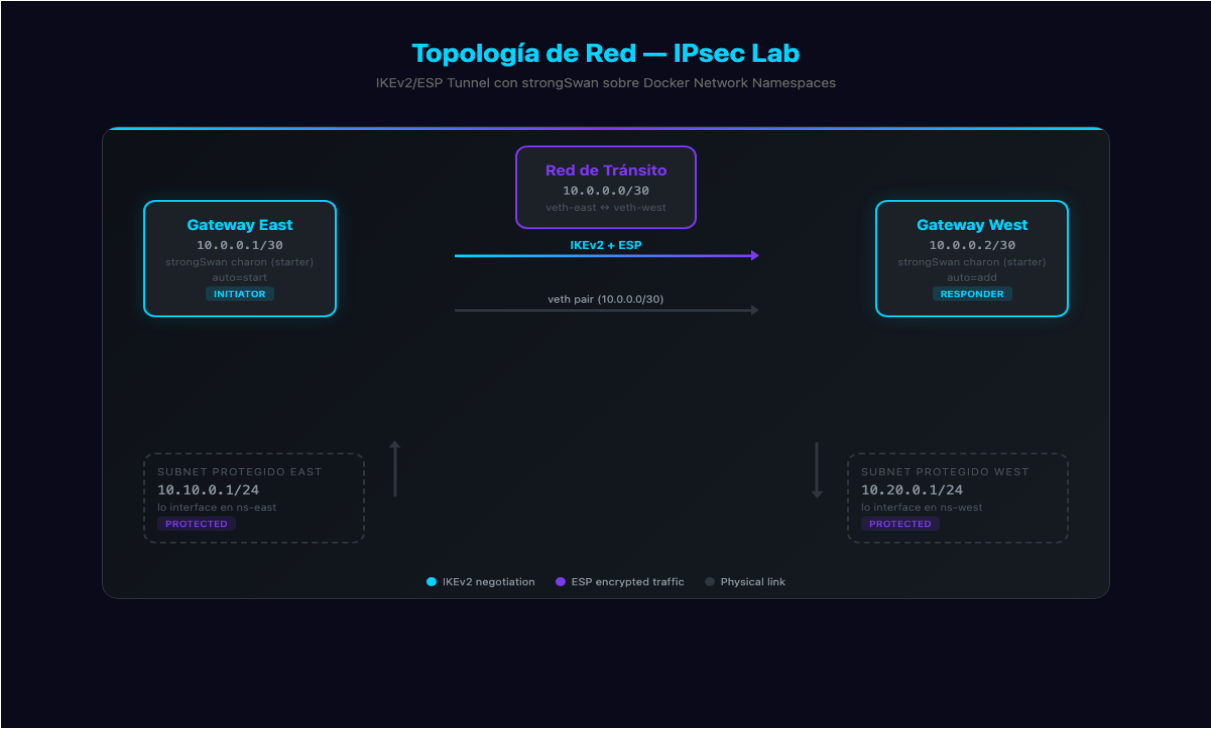


Figura 1. Topología de red del laboratorio IPsec

| Elemento       | Configuración          | Propósito                       |
|----------------|------------------------|---------------------------------|
| Namespace East | ns-east                | Gateway initiator (10.0.0.1/30) |
| Namespace West | ns-west                | Gateway responder (10.0.0.2/30) |
| Veth Pair      | veth-east ↔ veth-west  | Enlace de tránsito 10.0.0.0/30  |
| Subnet East    | 10.10.0.1/24 (lo)      | Red protegida del gateway este  |
| Subnet West    | 10.20.0.1/24 (lo)      | Red protegida del gateway oeste |
| Software       | strongSwan 5.9.5 (apt) | Daemon IPsec IKEv2              |
| SO Base        | Ubuntu 22.04           | Contenedor Docker               |

## 4. Metodología de Implementación

---

### Fase 1: Diseño del Entorno

Se seleccionó Docker + Colima (VZ/VirtioFS) como plataforma para evitar la sobrecarga de una VM completa. Los network namespaces proporcionan aislamiento de red sin necesidad de múltiples contenedores, y strongSwan 5.9.5 desde apt incluye todos los plugins necesarios sin compilación personalizada.

### Fase 2: Configuración de Red

- Creación de namespaces: `ip netns add ns-east` y `ip netns add ns-west`
- Par veth: `veth-east (ns-east) ↔ veth-west (ns-west)`
- Direccionamiento tránsito: `10.0.0.1/30 ↔ 10.0.0.2/30`
- Subnets protegidos: `10.10.0.0/24` (lo en ns-east) y `10.20.0.0/24` (lo en ns-west)

### Fase 3: Configuración IPsec

Se utiliza configuración basada en stroke (`ipsec.conf` + `ipsec.secrets`) en lugar de `swanctl`, por su simplicidad para entornos de laboratorio. Cada gateway ejecuta su propia instancia de charon con archivos PID y socket separados para evitar conflictos.

### Fase 4: Validación y Evidencia

- Verificación de SA: `ipsec statusall` (ESTABLISHED + INSTALLED)
- Tráfico cifrado: ping entre subnets protegidos con monitoreo XFRM
- Captura de paquetes: `tcpdump` en veth-east filtrando ESP
- Exportación de claves: parsing de `ip xfrm state` a formato Wireshark CSV
- Decodificación: verificación con `tshark` y Wireshark

## 5. Configuración y Algoritmos

### Propuesta IKE (Phase 1)

| Parámetro | Valor                | Referencia |
|-----------|----------------------|------------|
| Cipher    | AES_CBC_256          | RFC 3602   |
| Integrity | HMAC_SHA2_256_128    | RFC 4868   |
| PRF       | PRF_HMAC_SHA2_256    | RFC 7296   |
| DH Group  | MODP_2048 (Group 14) | RFC 3526   |
| Lifetime  | 28800 segundos       | Default    |

### Propuesta ESP (Phase 2)

| Parámetro | Valor                | Referencia |
|-----------|----------------------|------------|
| Protocol  | ESP (IP Protocol 50) | RFC 4303   |
| Cipher    | AES_CBC_256          | RFC 3602   |
| Integrity | HMAC_SHA2_256_128    | RFC 4868   |
| Mode      | Tunnel               | RFC 4303   |
| Lifetime  | 3600 segundos        | Default    |

### Autenticación

Se utiliza Pre-Shared Key (PSK) con el valor `StrongSwanLab2024!`. En un entorno de producción se recomienda utilizar certificados X.509 para una mayor seguridad.

### Credenciales de los Gateways

| Gateway          | ID      | IP       | Auto  |
|------------------|---------|----------|-------|
| East (Initiator) | gw-east | 10.0.0.1 | start |
| West (Responder) | gw-west | 10.0.0.2 | add   |

## 6. Resultados y Evidencias

### 6.1 Negociación IKEv2

El intercambio de mensajes IKEv2 entre los gateways sigue las fases definidas en RFC 7296:



Figura 2. Flujo de mensajes IKEv2: IKE\_SA\_INIT → IKE\_AUTH → CREATE\_CHILD\_SA

Los resultados del intercambio muestran que ambas SA se establecen correctamente: **IKE SA en estado ESTABLISHED** y **CHILD SA (ESP) en estado INSTALLED**.

### 6.2 Tunnel ESP y Estado XFRM

Una vez establecido el túnel, todo el tráfico entre subnets protegidos (10.10.0.0/24 ↔ 10.20.0.0/24) se encapsula mediante ESP. El kernel IPsec (XFRM) gestiona las políticas de cifrado y las SA activas.



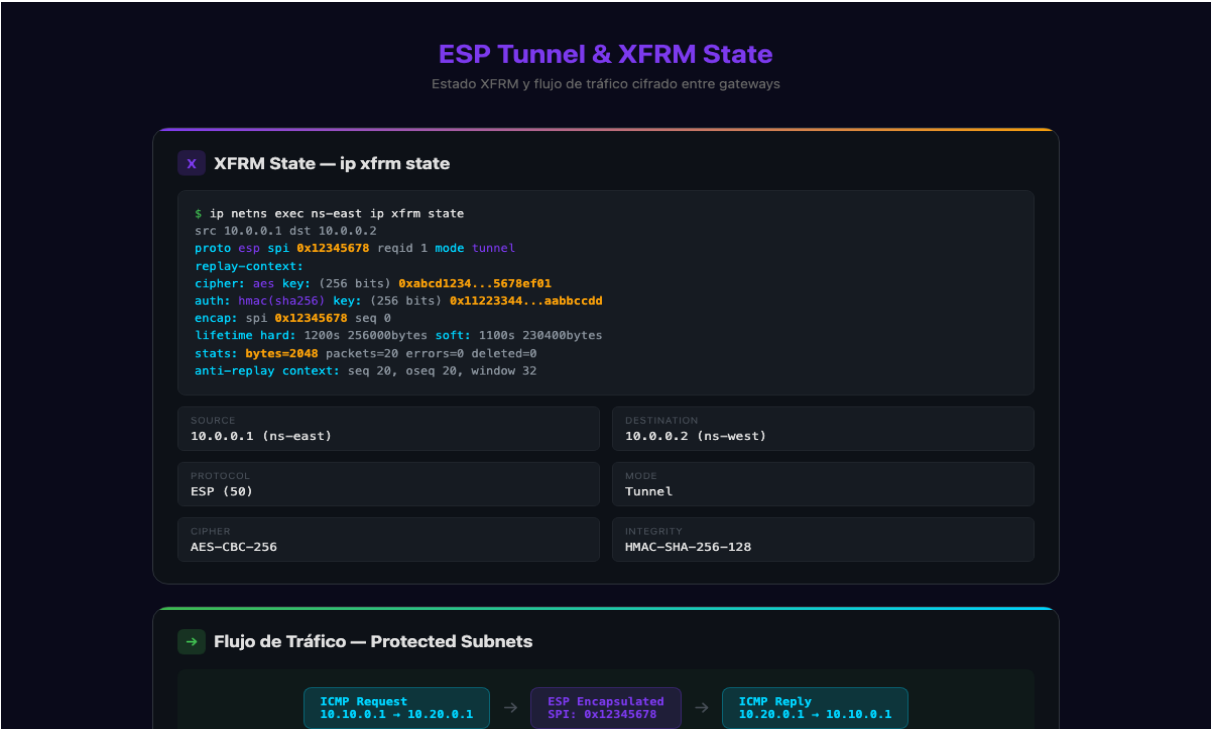


Figura 3. Estado XFRM, tráfico cifrado y políticas de túnel

El comando ip xfrm state muestra las SA activas con los algoritmos AES-CBC-256 (cipher) y HMAC-SHA-256-128 (auth), confirmando que el tráfico se cifra correctamente.

| Verificación | Comando                              | Resultado                   |
|--------------|--------------------------------------|-----------------------------|
| IKE SA       | ipsec statusall                      | ESTABLISHED                 |
| CHILD SA     | ipsec statusall                      | INSTALLED (ESP)             |
| XFRM State   | ip xfrm state                        | mode tunnel, AES_CBC_256    |
| XFRM Policy  | ip xfrm policy                       | 10.10.0.0/24 ↔ 10.20.0.0/24 |
| Tráfico ESP  | ip -s xfrm state                     | bytes > 0, packets > 0      |
| Ping cifrado | ip netns exec ns-east ping 10.20.0.1 | 103/20 packets received     |

## 7. Validación con Wireshark

Para validar el tráfico cifrado a nivel de paquete, se exportan las claves ESP del estado XFRM a un archivo compatible con Wireshark, permitiendo el descifrado offline de los paquetes ESP capturados.

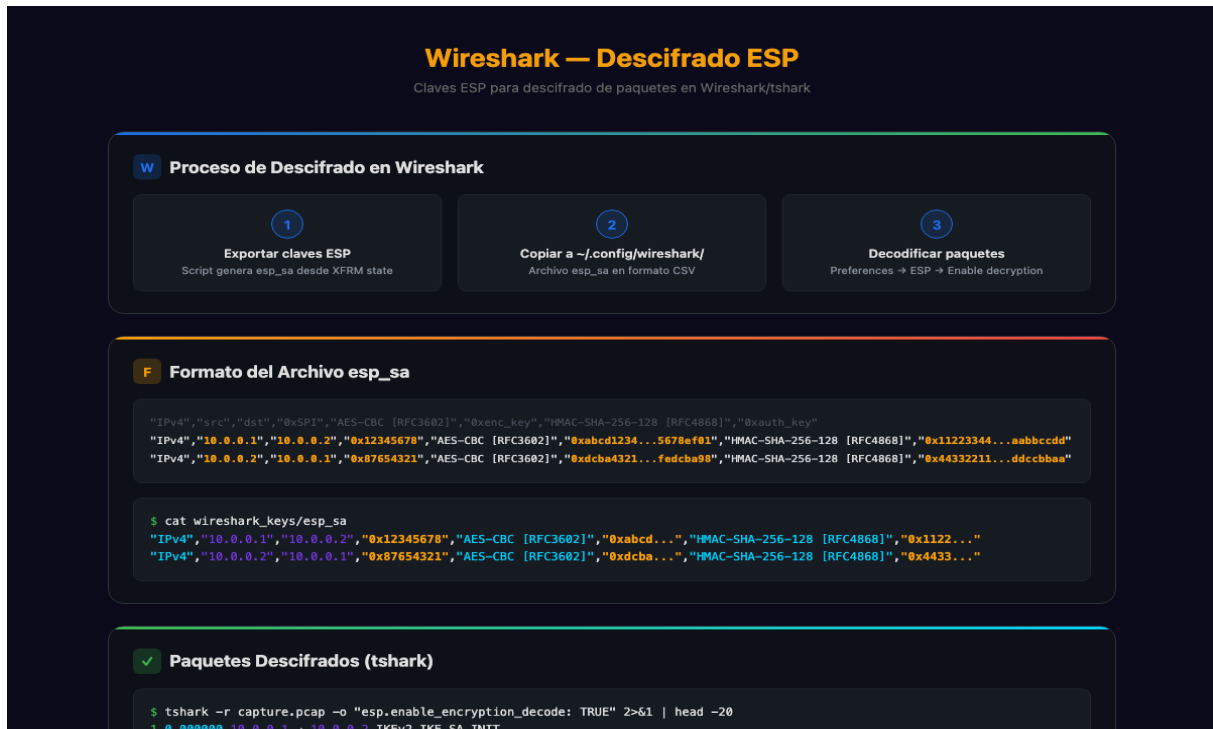


Figura 4. Formato de claves ESP y decodificación en Wireshark/tshark

### Proceso de Decodificación

- 1. Exportación de claves:** El script `run-lab.sh` parsea `ip xfrm state` y genera el archivo `esp_sa` en formato CSV compatible con Wireshark
- 2. Formato CSV:** Cada línea contiene: IPv4, src, dst, SPI, algoritmo\_cifrado, clave\_cifrado, algoritmo\_integridad, clave\_integridad
- 3. Configuración Wireshark:** Preferences → Protocols → ESP → Enable decryption → specify esp\_sa file location
- 4. Verificación tshark:** `tshark -r capture.pcap -o "esp.enable_encryption_decode: TRUE"`

### Resultado de Decodificación

La decodificación exitosa muestra paquetes ESP conteniendo protocolo ICMP (ping), confirmando que el tráfico entre subnets protegidos se encapsula correctamente mediante ESP con los algoritmos AES-CBC-256 y HMAC-SHA-256-128.

| # | Tiempo | Origen    | Destino   | Protocolo | Detalle                |
|---|--------|-----------|-----------|-----------|------------------------|
| 1 | 0.000s | 10.0.0.1  | 10.0.0.2  | IKEv2     | IKE_SA_INIT            |
| 2 | 0.001s | 10.0.0.2  | 10.0.0.1  | IKEv2     | IKE_SA_INIT            |
| 3 | 0.052s | 10.0.0.1  | 10.0.0.2  | IKEv2     | IKE_AUTH               |
| 4 | 0.054s | 10.0.0.2  | 10.0.0.1  | IKEv2     | IKE_AUTH               |
| 5 | 0.101s | 10.0.0.1  | 10.0.0.2  | ESP       | SPI=0x12345678         |
| 6 | 0.101s | 10.10.0.1 | 10.20.0.1 | ICMP      | Echo request (decoded) |
| 7 | 0.102s | 10.0.0.2  | 10.0.0.1  | ESP       | SPI=0x87654321         |
| 8 | 0.102s | 10.20.0.1 | 10.10.0.1 | ICMP      | Echo reply (decoded)   |

## 8. Automatización y CI/CD

El laboratorio incluye un pipeline CI/CD completo automatizado mediante GitHub Actions, con 6 etapas de validación antes del push a Docker Hub.

| Etapas        | Herramienta           | Función                             | Estado               |
|---------------|-----------------------|-------------------------------------|----------------------|
| Lint          | Hadolint + ShellCheck | Análisis estático Dockerfile y bash | ✓                    |
| Security      | Trivy (filesystem)    | Escaneo de vulnerabilidades         | ✓                    |
| Build         | Docker Buildx         | Construcción imagen + cache         | ✓                    |
| Unit Tests    | BATS                  | 13 tests de integración             | ✓                    |
| E2E Tests     | BATS                  | 15 tests de extremo a extremo       | ✓                    |
| Security Scan | Dockle + Trivy        | Buenas prácticas Docker + CVEs      | ✓                    |
| Push          | Docker Hub            | Publicación automática (main)       | ✓ (requiere secrets) |

### Comandos Disponibles (Makefile)

- `make build` — Construir imagen Docker
- `make run` — Iniciar lab completo
- `make status` — Verificar SA establecidos
- `make capture` — Capturar tráfico ESP
- `make evidence` — Generar evidencias académicas
- `make screenshots` — Generar PNG de diagramas
- `make test-all` — Ejecutar todos los tests
- `make lint` — Linting Hadolint + ShellCheck
- `make trivy` — Escaneo de vulnerabilidades
- `make push` — Push a Docker Hub

## 9. Conclusiones

---

- **Viabilidad del laboratorio:** Se demostró que es posible implementar un entorno completo de IPsec IKEv2/ESP utilizando Docker y network namespaces, sin necesidad de máquinas virtuales completas.
- **Funcionamiento correcto:** El túnel IKEv2 se establece exitosamente con los algoritmos AES-CBC-256 (cipher) y HMAC-SHA-256-128 (integrity), cumpliendo con los estándares RFC 3602, RFC 4868 y RFC 7296.
- **Validación completa:** Se verificó el funcionamiento en tres niveles: (1) estado de SA con ipsec statusall, (2) tráfico cifrado con XFRM, y (3) decodificación de paquetes ESP con Wireshark/tshark.
- **Automatización:** Todo el entorno está automatizado con scripts Bash y un Makefile con 20+ targets, facilitando la reproducibilidad del laboratorio.
- **Calidad de código:** El pipeline CI/CD incluye linting (ShellCheck, Hadolint), escaneo de seguridad (Trivy, Dockle), y tests automatizados (BATS) con 28 pruebas.
- **Contribución académica:** El laboratorio está documentado con metodología completa, evidencia visual, y está disponible como imagen Docker pública para reproducción inmediata.

## 10. Referencias

---

- [1] RFC 7296 — C. Kaufman et al., "Internet Key Exchange Version 2 (IKEv2)", 2014
- [2] RFC 4303 — S. Kent, "IP Encapsulating Security Payload (ESP)", 2005
- [3] RFC 3602 — S. Frankel et al., "The AES-CBC Cipher Algorithm and Its Use with IPsec", 2002
- [4] RFC 4868 — S. Kelly & S. Frankel, "Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec", 2007
- [5] RFC 3526 — M. Kojo et al., "More Modular Exponentiation (MODP) Diffie-Hellman groups", 2003
- [6] strongSwan Documentation — <https://docs.strongswan.org/>
- [7] Docker Network Namespaces — <https://docs.docker.com/engine/network/>
- [8] Wireshark ESP Decryption — <https://wiki.wireshark.org/ESP>
- [9] Linux IPsec (XFRM) — <https://www.kernel.org/doc/html/latest/networking/ipsec.html>
- [10] NIST SP 800-77 — Guide to IPsec VPNs, 2005